

[APRENDE REACT >](#)

Inicio rápido

¡Bienvenido a la documentación de React! Esta página te dará una introducción a un 80% de los conceptos de React que usarás a diario.

Aprenderás

- Cómo crear y anidar componentes
- Cómo añadir marcado y estilos
- Cómo mostrar datos
- Cómo renderizar condicionales y listas
- Cómo responder a eventos y actualizar la pantalla
- Cómo compartir datos entre componentes

Crear y anidar componentes

Las aplicaciones de React están hechas a partir de *componentes*. Un componente es una pieza de UI (siglas en inglés de interfaz de usuario) que tiene su propia lógica y apariencia. Un componente puede ser tan pequeño como un botón, o tan grande como toda una página.

Los componentes de React son funciones de JavaScript que devuelven *markup* (marcado):

```
function MyButton() {  
  return (  
    <button>Soy un botón</button>  
  );  
}
```

Ahora que has declarado `MyButton`, puedes anidarlo en otro componente:

```
export default function MyApp() {  
  return (  
    <div>  
      <h1>Bienvenido a mi aplicación</h1>  
      <MyButton />  
    </div>  
  );  
}
```

Nota que `<MyButton />` empieza con mayúscula. Así es como sabes que es un componente de React. Los nombres de los componentes de React siempre deben comenzar con mayúscula, mientras las etiquetas HTML deben estar minúsculas.

Mira el resultado:

App.js

⬇ Descargar ↻ Reiniciar 🔗

```
function MyButton() {  
  return (  
    <button>  
      Soy un botón  
    </button>  
  );  
}  
  
export default function MyApp() {  
  return (  
    <div>  
      <h1>Bienvenido a mi aplicación</h1>
```

▼ Mostrar más

Bienvenido a mi aplicación

Soy un botón

Las palabras clave `export default` especifican el componente principal en el archivo. Si no estás familiarizado con alguna parte de la sintaxis de JavaScript, [MDN](#) y [javascript.info](#) tienen magníficas referencias.

Escribir marcado con JSX

La sintaxis de marcado que viste arriba se llama *JSX*. Es totalmente opcional, pero la mayoría de los proyectos de React usan JSX por la comodidad que ofrece. Todas las [herramientas que recomendamos para el desarrollo local](#) son compatibles con JSX sin ningún tipo de configuración.

JSX es más estricto que HTML. Tienes que cerrar etiquetas como `<br />`. Tu componente tampoco puede devolver múltiples etiquetas de JSX. Debes envolverlas en un padre compartido, como `<div>...</div>` o en un envoltorio vacío `<>...</>`:

```
function AboutPage() {  
  return (  
    <>  
      <h1>Acerca de</h1>  
    </>  
  );  
}
```

```
<p>Hola.<br />¿Cómo vas?</p>  
</>  
>;  
>
```

Si tienes mucho HTML que convertir a JSX, puedes utilizar un [convertidor en línea](#).

Añadir estilos

En React, especificas una clase de CSS con `className`. Funciona de la misma forma que el atributo `class` de HTML:

```
<img className="avatar" />
```

Luego escribes las reglas CSS para esa clase en un archivo CSS aparte:

```
/* In your CSS */  
.avatar {  
  border-radius: 50%;  
}
```

React no prescribe como debes añadir tus archivos CSS. En el caso más simple, añades una etiqueta `<link>` a tu HTML. Si utilizas una herramienta de construcción o un framework, consulta su documentación para saber como añadir un archivo CSS a tu proyecto.

Mostrar datos

JSX te permite poner marcado dentro de JavaScript. Las llaves te permiten “escapar de nuevo” hacia JavaScript de forma tal que puedas incrustar una variable de tu código y mostrársela al usuario. Por ejemplo, esto mostrará `user.name`:

```
return (  
  <h1>  
    {user.name}  
  </h1>  
)  
);
```

También puedes “escaparte hacia JavaScript” en los atributos JSX, pero tienes que utilizar llaves *en lugar de* comillas. Por ejemplo, `className="avatar"` pasa la cadena "avatar" como la clase CSS, pero `src={user.imageUrl}` lee el valor de la variable de JavaScript `user.imageUrl` y luego pasa el valor como el atributo `src`:

```
return (  
  <img  
    className="avatar"  
    src={user.imageUrl}  
  />  
)  
);
```

Puedes también poner expresiones más complejas dentro de llaves, por ejemplo, [concatenación de cadenas](#):

App.js

⬇ Descargar ⌂ Reiniciar ↗

```
const user = {  
  name: 'Hedy Lamarr',  
  imageUrl: 'https://i.imgur.com/yX0vd0Ss.jpg',  
  imageSize: 90,  
};  
  
export default function Profile() {  
  return (  
    <>  
      <h1>{user.name}</h1>  
      <img  
        className="avatar"
```